

Technical Analysis of Reconstructed MATLAB Evolutionary-Optimization Workflows

Faithful Python reconstruction of Evolution Strategy and Traveling
Salesman Problem workflows

Publication-oriented technical report

Source analyzed: `matlab_reconstructed_workflows(1).py`

Contents

Abstract	1
1 Introduction	1
1.1 Scope and source basis	2
2 Project Overview	2
3 Technical Foundations	3
3.1 Evolutionary search	3
3.2 Evolution Strategies and self-adaptation	3
3.3 Ackley objective	4
3.4 Permutation representations for the TSP	4
4 Data and Input Processing	4
4.1 Continuous ES inputs	4
4.2 TSP file parsing	5
4.3 Input validation	5
5 System Architecture and Code Organization	5
5.1 Reproducibility layer	5
5.2 ES representation and operators	5
5.3 TSP representation and operators	6
6 Detailed Methodology	6
6.1 Evolution Strategy initialization	6
6.2 ES crossover	7
6.3 ES mutation and bounds	7
6.4 ES survivor selection and stopping	8
6.5 Repeated ES execution	8
6.6 TSP cost and fitness	8
6.7 Permutation crossover	8
6.8 Permutation mutation	9
6.9 Parent selection	9
6.10 TSP survivor replacement	9
6.11 TSP stopping and repeated workflows	9

7 Mathematical Formulation	10
7.1 Unified ES view	10
7.2 TSP as permutation optimization	10
7.3 Exact validation on small TSP instances	10
8 Optimization and Learning Procedure	11
8.1 Initialization and scale adaptation	11
8.2 Selection pressure and replacement	11
8.3 Objective thresholds	11
9 Implementation Details	11
9.1 Software dependencies	11
9.2 Copy semantics and mutable state	12
9.3 Serialization	12
9.4 Colab portability	12
10 Execution Workflow	12
11 Design Decisions and Rationale	13
12 Computational Considerations	13
12.1 Evolution Strategy cost	13
12.2 TSP cost	14
12.3 Memory behavior	14
13 Reproducibility and Reliability	14
14 Limitations and Technical Considerations	15
14.1 Syntactic execution defect in the submitted file	15
14.2 Correlation state without covariance rotation	15
14.3 Terminology encoded as identifiers	15
14.4 Distance convention and TSPLIB semantics	16
14.5 Fitness reciprocity	16
14.6 No complete dependency environment specification	16
15 Overall Technical Assessment	16
16 Conclusion	17

Abstract

The analyzed source file reconstructs four optimization workflows that were originally organized as MATLAB programs: two Evolution Strategy (ES) workflows for a continuous objective and two permutation-based evolutionary workflows for the Traveling Salesman Problem (TSP). The implementation combines the reconstructed algorithms with runtime metadata, deterministic seeding, validation utilities, artifact serialization, and optional Google Colab download behavior. It therefore serves both as an optimization implementation and as a reproducibility-oriented harness around it.

The main technical contribution is not a new optimization algorithm, but a careful translation of several evolutionary-computation mechanisms into Python while preserving configurable choices visible in the reconstructed MATLAB workflows. The ES implementation supports uncorrelated one-step mutation, uncorrelated n -step mutation, and a correlated mutation representation; several crossover modes are available; survivor selection distinguishes (μ, λ) and $(\mu + \lambda)$ replacement; and the primary objective is the Ackley function, with a shifted Rosenbrock objective retained as an alternative from the source archive. The TSP implementation uses permutation representations, several permutation-safe crossover operators, four mutation operators, multiple parent-selection policies, fitness defined as the reciprocal of tour cost, and evolutionary survivor replacement.

This report reconstructs the conceptual and computational pipeline from the source. It separates general theory from implementation details, states the mathematical formulations explicitly, analyzes complexity and numerical behavior, and identifies key fidelity decisions and limitations. It also distinguishes the intended algorithmic structure from the literal execution behavior of the submitted file. One significant issue is that the file places a `from __future__ import annotations` statement after additional top-level string literals, so it is not directly executable by Python without a syntactic adjustment. That issue is documented rather than hidden. No experimental figures or result visualizations are reproduced here.

1 Introduction

The project addresses a classical computational-optimization setting: search over large or non-convex spaces where exact deterministic optimization is difficult or impractical for the intended use. It has two complementary targets. The Evolution Strategy component operates on real-valued vectors and evolves both decision variables and mutation-control parameters. The TSP component operates on permutations and therefore requires operators that preserve route validity rather than treating routes as unconstrained numeric vectors.

The file describes itself as a “faithful Python reconstruction” of the supplied MATLAB ES and TSP workflows. This framing is important when interpreting the implementation: the primary engineering goal is fidelity to an existing workflow, not redesign around a modern Python optimization framework. The reconstruction therefore retains configuration names and operator combinations from the MATLAB programs, including spellings such as `tournament`, `roletwheel`, `trancation`, and `landa_va_mua`. These are implementation identifiers, not standard terminology.

The report covers the complete file, including environment setup, reproducibility utilities,

objective functions, population representations, genetic operators, validation routines, result serialization, workflow dispatch, and the final executable cells. The emphasis is on what the code actually does and what follows from that behavior. Where intent is not explicitly encoded, the interpretation is presented as an inference rather than as a fact.

1.1 Scope and source basis

The source is a linearized Python form of a Colab notebook. Its opening lines identify the original notebook name and a Colab location, while later blocks use section-like triple-quoted strings as notebook-style headings. The file contains 1,257 lines and defines 47 functions and four data classes. NumPy, pandas, and Matplotlib are the main numerical and export dependencies; SciPy and PyTorch are probed optionally in the runtime metadata. The four workflows are explicitly listed in `WORKFLOW_INVENTORY`.

The report does not assume access to the original MATLAB source archives because those archives are not present in the uploaded file. The reconstruction does preserve path names such as `ES_Eshraghi_Sharifi/code/main.m` and `TSP_Eshraghi_Sharifi/code/main.m`, which provide provenance context. Claims about the original programs are therefore limited to what the reconstructed file explicitly records.

2 Project Overview

Conceptually, the program is organized into four layers. The first is the optimization core: continuous ES routines and permutation-based TSP routines. The second is validation, which checks finite outputs, route validity, exact optimality on a small synthetic TSP instance, and optional forensic consistency of supplied output files. The third is reproducibility and artifact management, which records software versions, seeds, and runtime platform information and exports JSON, CSV, and ZIP artifacts. The fourth is workflow orchestration, which exposes bounded validation settings as defaults while retaining the larger original-scale configurations for reference.

Workflow	Optimization problem	Core configuration exposed by the source
1	Continuous ES, single run	Correlated mutation; average-discrete crossover; (μ, λ) survivor mode; Ackley objective
2	Continuous ES, repeated runs	Same ES configuration; repeated independent seeds; iteration summary
3	TSP, single run	Order-1 crossover; swap mutation; tournament parent selection; (μ, λ) survivor mode
4	TSP, repeated runs	Order-1 crossover; inversion mutation; tournament parent selection; repeated independent seeds

Table 1: Workflow inventory reconstructed from the source configuration.

A central design pattern is the separation of configuration from execution. `ESConfig` and `TSPConfig` collect algorithmic parameters, while `run_es` and `run_tsp` implement the iterative search. The same engines can therefore serve both single-run and repeated-run workflows.

The data flow is straightforward. For ES, an initial population is generated, each member is evaluated, children are created by crossover, mutated, and re-evaluated, survivors are selected, and the best cost is checked against a threshold. For TSP, random permutations initialize the population, route costs are converted to reciprocal fitness, parents are selected, permutation crossover creates children, mutation is applied, survivor replacement forms the next population, and the best fitness and route are recorded.

3 Technical Foundations

3.1 Evolutionary search

Evolutionary algorithms maintain a population rather than a single iterate. Let $P_t = \{x_1, \dots, x_\mu\}$ denote a population of size μ . A generic generation constructs offspring C_t from P_t , evaluates them under an objective or fitness function, and forms P_{t+1} either from the offspring alone or from the union of parents and offspring. The implementation supports both replacement patterns through the survivor-mode switch.

For a minimization objective $f(x)$, lower values are preferred. In the TSP implementation, fitness is instead defined as the reciprocal of cost, so larger values are preferred:

$$F(\pi) = \frac{1}{C(\pi)}, \quad (1)$$

where π is a permutation and $C(\pi)$ is its closed-tour length. This is a simple monotonic transformation whenever $C(\pi) > 0$, so maximizing F is equivalent to minimizing C .

3.2 Evolution Strategies and self-adaptation

An ES population member is a pair (x, η) , where $x \in \mathbb{R}^d$ is the decision vector and η stores mutation-control information. This is a self-adaptive representation because the strategy parameters evolve with the solution variables.

The general mutation form is

$$x' = \text{clip}(x + \varepsilon, \ell, u), \quad \varepsilon \sim \mathcal{N}(0, \Sigma), \quad (2)$$

where ℓ and u are componentwise lower and upper bounds and the covariance matrix Σ depends on the selected mutation family.

For the uncorrelated one-step representation, one scalar step size σ is maintained and

$$\Sigma = \sigma I_d. \quad (3)$$

For the uncorrelated n -step representation, a vector $\sigma \in \mathbb{R}_{>0}^d$ is maintained and

$$\Sigma = \text{diag}(\sigma). \quad (4)$$

The correlated representation additionally stores an antisymmetric angle-like matrix α , but the submitted code does not use α to rotate the covariance matrix when drawing mutation noise. This distinction between representation and operational effect is important and is

discussed later.

3.3 Ackley objective

The primary ES objective is the standard Ackley form used by the implementation:

$$f(x) = -20 \exp \left(-0.2 \frac{\|x\|_2}{\sqrt{d}} \right) - \exp \left(\frac{1}{d} \sum_{i=1}^d \cos(2\pi x_i) \right) + 20 + e. \quad (5)$$

The code flattens its input, computes the Euclidean norm, averages the cosine term, and returns a scalar floating-point objective. This yields a continuous, multimodal benchmark whose global minimum is at the origin. The report does not infer performance beyond what the algorithm and validation code support.

The implementation also includes a shifted Rosenbrock objective,

$$f(x) = 200 + \sum_{i=1}^{d-1} \left[100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2 \right], \quad (6)$$

which is not used by the default workflow dispatcher. Its presence indicates that the reconstruction retained an alternative objective visible in the source archive.

3.4 Permutation representations for the TSP

A TSP solution is represented by a permutation $\pi = (\pi_1, \dots, \pi_n)$ containing each city index exactly once. The route is closed by joining the last city back to the first. The `tsp_cost` function computes

$$C(\pi) = \sum_{i=1}^n d_{\pi_i, \pi_{i+1}}, \quad \pi_{n+1} = \pi_1. \quad (7)$$

A distance value of -1 is replaced with $+\infty$, so that entry is treated as a prohibited edge. The Euclidean distance matrix constructor uses

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}. \quad (8)$$

4 Data and Input Processing

4.1 Continuous ES inputs

The ES routines do not require an external dataset. The population is generated directly from a uniform distribution over a box:

$$x_i \sim \text{Uniform}(\ell, u), \quad i = 1, \dots, d. \quad (9)$$

The benchmark dimension, variable bounds, population size, crossover and mutation probabilities, offspring multiplier, survivor rule, mutation family, and stopping threshold are encoded in `ESConfig`.

The validation configuration is deliberately separated from the original-scale configuration.

The validation path uses small dimensions and iteration counts to keep the end-to-end run bounded, while the original values are preserved in a JSON configuration object. This allows the notebook to exercise the full control flow without automatically launching the much larger original search.

4.2 TSP file parsing

The TSP parser reads a TSPLIB-style text file line by line. It extracts the instance name, dimension, edge-weight type, and coordinate section. It recognizes the `NODE_COORD_SECTION` marker and parses each coordinate row into a floating-point pair. If the declared dimension does not match the number of parsed coordinates, an exception is raised.

A separate helper, `load_distance_matrix_text`, handles a text format in which the square matrix appears after five header values. The helper is part of the reconstruction but is not connected to the default execution path shown at the bottom of the file. The active validation example instead hard-codes a six-city matrix described in the notebook text as the exact matrix from `data.txt`.

4.3 Input validation

The TSP route validator checks route length, whether the sorted route equals the full set of city identifiers $\{0, \dots, n - 1\}$, and the number of duplicate identifiers. This is a structural validity check rather than a proof of optimality. For small instances, the source additionally provides an exact brute-force optimizer, which fixes city 0 as the starting point to eliminate rotational duplicates.

5 System Architecture and Code Organization

The implementation is best understood as a single-module research script with clearly separated conceptual regions.

5.1 Reproducibility layer

`set_global_seed` seeds both Python's standard `random` module and NumPy's legacy global RNG. Individual workflows additionally use `numpy.random.Generator` instances created from explicit seeds. This dual pattern is useful when code paths mix random APIs, although the main optimization operators use the local generators.

`runtime_metadata` records the Python version, platform, NumPy, pandas, and Matplotlib versions, plus optional SciPy and PyTorch information. For PyTorch it also records CUDA availability and, when available, the first GPU name. These values are metadata for reproducibility rather than dependencies of the optimization algorithm itself.

5.2 ES representation and operators

`ESPopulationMember` bundles three fields: the solution vector, its strategy parameter representation, and its current cost. This avoids parallel arrays and allows the solution and

strategy state to be copied together.

`initialize_es_population` creates initial solution vectors and initializes the corresponding strategy parameters. The scalar and vector mutation families use log-normal sampling; the correlated mode stores both positive scale parameters and an antisymmetric matrix.

`crossover_es` supports four modes: discrete, average, average-discrete, and discrete-average. These modes independently combine solution coordinates and strategy parameters. In correlated mode, the code handles the scale vector and the α matrix separately. The use of copied parameters prevents accidental sharing of mutable NumPy arrays between parent and child representations.

`mutate_es` applies mutation to each candidate with probability p_m . The code draws multivariate normal noise from a covariance matrix derived from the evolving step-size state, clips the mutated variable to the configured box constraints, and resets the child's cost to zero until re-evaluation.

`select_es_survivors` implements two population-replacement policies. In the `landa+mua` case, parents and children compete jointly. In the `landa_va_mua` case, only the children form the survivor pool. Despite their nonstandard identifiers, these correspond conceptually to $(\mu + \lambda)$ and (μ, λ) replacement schemes, respectively.

5.3 TSP representation and operators

The TSP side uses NumPy arrays with integer city identifiers. Four crossover operators are implemented: PMX, Order-1, Cycle, and MOX. Four mutation operators are implemented: swap, insert, inversion, and scramble. Parent selection supports tournament, roulette-style, truncation, and full-random selection, while survivor selection can use parent-plus-child competition or child-only replacement.

This modularity keeps the core evolutionary loop independent of any particular permutation operator. The selected mode is routed through dictionaries or branches, and invalid mode names raise explicit exceptions.

6 Detailed Methodology

6.1 Evolution Strategy initialization

For each individual, x is sampled uniformly from the configured search box, and the strategy parameters are initialized according to the selected mutation family. A soft lower floor is applied through

$$\text{softfloor}(z; \epsilon) = \max(z - \epsilon, 0) + \epsilon. \quad (10)$$

For nonnegative z , this keeps the result at least ϵ while becoming the identity once $z \geq 2\epsilon$. The implementation uses $\epsilon = 0.01$. The transform prevents zero or near-zero mutation scales from eliminating exploration.

For the one-step method, the code uses

$$\tau = \frac{1}{\sqrt{d}}, \quad \sigma' = \text{softfloor}(\text{LogNormal}(0, \tau^2); \epsilon). \quad (11)$$

For the n -step and correlated initializations, it uses

$$\tau = \frac{1}{\sqrt{2\sqrt{d}}}, \quad \tau' = \frac{1}{\sqrt{2d}}, \quad (12)$$

and combines a common log-normal factor with componentwise log-normal factors.

6.2 ES crossover

For each child, the ES crossover routine samples two parents independently from the current population. The child initially copies the first parent, after which crossover is applied with probability p_c . The four supported modes differ in whether solution coordinates and strategy parameters are inherited discretely or averaged.

For example, average crossover for the decision vector gives

$$x^{(c)} = \frac{x^{(1)} + x^{(2)}}{2}. \quad (13)$$

Discrete crossover uses Bernoulli masks $b_i \in \{0, 1\}$ and

$$x_i^{(c)} = b_i x_i^{(1)} + (1 - b_i) x_i^{(2)}. \quad (14)$$

The source extends the same idea to strategy parameters, using a matrix-valued antisymmetry mask for the correlated α representation so that the resulting matrix remains antisymmetric.

6.3 ES mutation and bounds

Each child is mutated independently with probability p_m . For the one-step model,

$$\Sigma = \sigma' I_d, \quad (15)$$

whereas for the n -step model,

$$\Sigma = \text{diag}(\boldsymbol{\sigma}'). \quad (16)$$

The correlated branch updates the scale vector and angle-like matrix using log-normal and Gaussian perturbations, respectively. The code then constructs *only* $\text{diag}(\boldsymbol{\sigma}')$ as the covariance passed to the multivariate normal sampler.

This is a key fidelity detail. The implementation explicitly preserves α , but it does not construct a rotated covariance matrix from α . Therefore the operational mutation distribution in the submitted Python code is still diagonal. The representation is “correlated” in the sense that angle parameters are maintained and mutated, but the current implementation does not use those angles to generate correlated Gaussian directions. A reader should not describe the code as implementing a full covariance-rotation ES without qualification.

The final variable update is clipped:

$$x' = \min\{u, \max\{\ell, x + \varepsilon\}\}, \quad (17)$$

componentwise. Clipping guarantees box feasibility after each mutation but also changes the

effective sampling distribution near the boundary.

6.4 ES survivor selection and stopping

Given population size μ and offspring count λ , the code sorts candidates by objective value and keeps the best μ . For (μ, λ) replacement only the offspring compete; for $(\mu + \lambda)$ replacement parents are included. The best objective after each generation is recorded, and execution terminates early if

$$f(x_{\text{best}}) \leq \tau_f, \quad (18)$$

where the default threshold is 0.1.

6.5 Repeated ES execution

The repeated ES workflow calls the single-run engine with seeds $s, s + 1, \dots, s + r - 1$. It collects the number of iterations executed in each run, counts threshold-reaching runs, and reports the sample mean and sample standard deviation of the iteration counts. The standard deviation uses `ddof=1`, so it is the usual unbiased sample standard deviation when multiple runs are available.

6.6 TSP cost and fitness

The TSP route cost closes the tour explicitly by shifting the route one position with `np.roll`. If a route is π , the implementation evaluates every edge (π_i, π_{i+1}) with the wrap-around convention $\pi_{n+1} = \pi_1$. The fitness is then the reciprocal of the cost.

This choice makes high fitness equivalent to low route length, but it also introduces the expected numerical hazard at zero cost. The implementation guards against divide-by-zero warnings when computing fitness, yet the underlying reciprocal remains conceptually singular. For a valid non-degenerate TSP instance with positive inter-city distances, the normal operating regime is strictly positive cost.

6.7 Permutation crossover

The implementation includes multiple crossover operators because permutations require feasibility-preserving recombination.

PMX. A segment from one parent is copied directly and mapping relationships are used to fill the remainder. This preserves a permutation while maintaining position-level information from both parents.

Order-1 crossover. A segment from one parent is copied, and the remaining positions are filled in the cyclic order induced by the other parent, skipping values already present. This preserves relative order information.

Cycle crossover. The code constructs position cycles between the two parents and alternates parent sources across cycles. Because each value occurs exactly once in each parent, the cycle mapping is well-defined for valid permutations.

MOX. The implementation selects a single cut point, preserves the prefix from each parent, and completes each child with values from the opposite parent while skipping duplicates. This is a compact permutation-safe recombination scheme.

The workflow-level defaults use Order-1 crossover, but the general engine retains the other operators for comparative experimentation.

6.8 Permutation mutation

All mutation functions begin by selecting two cut points. Swap exchanges the selected positions. Insert moves one value into the interval beginning at the first cut. Inversion reverses the selected segment, while scramble randomly permutes that segment. These operators have different locality properties: inversion is structured and often changes a contiguous sub-tour, whereas scramble introduces greater local disruption.

6.9 Parent selection

Tournament selection samples $k = \lfloor 0.5n \rfloor$ candidates without replacement, with a minimum of one, and returns the candidate with maximum fitness. Roulette-style selection first shifts the fitness values by their maximum, exponentiates the shifted values, and normalizes them. Specifically,

$$w_i = \exp(\text{clip}(F_i - F_{\max}, -700, 0)), \quad p_i = \frac{w_i}{\sum_j w_j}. \quad (19)$$

The shift improves numerical stability by preventing unnecessary overflow. Truncation selection samples uniformly from the top half of the population, while full-random selection ignores fitness entirely.

The workflow configuration uses tournament selection. With a half-population tournament, selection pressure is substantial: the best individual among a comparatively large sampled subset is favored each time a parent is required. This can accelerate exploitation but also reduce diversity when combined with strong elitist replacement.

6.10 TSP survivor replacement

The TSP survivor function mirrors the ES design. In parent-plus-child mode, the candidate pool is the vertical concatenation of parents and offspring. In child-only mode, only offspring are available. The new population is then filled by repeatedly calling the configured parent-selection method on the candidate pool itself.

This is slightly different from deterministic “keep the best μ ” replacement: selection is stochastic because `select_one` is called repeatedly. Consequently, even when the candidate pool is fixed, repeated sampling can produce a new population that is not simply the top μ individuals. The exact behavior depends on the selected selection mode.

6.11 TSP stopping and repeated workflows

The main TSP loop records best fitness, best cost, and best route at every iteration. It may stop early if a fitness threshold is configured. The repeated workflow for the six-city

validation matrix uses such a threshold condition in addition to a route-cost based break. In the general engine, however, the stop criterion is expressed in terms of fitness, so the interpretation of any threshold must account for the reciprocal transformation.

7 Mathematical Formulation

7.1 Unified ES view

The ES can be summarized as a state transition over pairs (x, η) . Let $P_t = \{(x_i, \eta_i)\}_{i=1}^\mu$. Each generation creates λ children by selecting parent pairs, applying a crossover operator $\mathcal{C}$ with probability p_c , and applying a mutation operator $\mathcal{M}$ with probability p_m :

$$(x'_j, \eta'_j) = \mathcal{M}(\mathcal{C}((x_a, \eta_a), (x_b, \eta_b))). \quad (20)$$

The children are evaluated under f , and the survivor map $\mathcal{S}$ yields

$$P_{t+1} = \mathcal{S}(P_t, C_t). \quad (21)$$

This abstraction covers the concrete branching without suppressing the implementation-specific choices.

7.2 TSP as permutation optimization

For n cities, the search space contains $n!$ raw permutations, with rotational and sometimes reverse symmetries in the tour representation. The implementation does not explicitly canonicalize routes during evolutionary search. Instead, it relies on permutation-preserving operators and validates that candidate routes contain each city exactly once.

The objective is

$$\min_{\pi \in \mathfrak{S}_n} C(\pi), \quad (22)$$

where $\mathfrak{S}_n$ denotes the set of permutations of the n city identifiers. The code's reciprocal fitness converts this into the equivalent maximization problem

$$\max_{\pi \in \mathfrak{S}_n} F(\pi), \quad F(\pi) = C(\pi)^{-1}. \quad (23)$$

7.3 Exact validation on small TSP instances

The brute-force verifier enumerates permutations of cities $1, \dots, n-1$ while fixing city 0 as the first city. Its worst-case enumeration count is

$$(n-1)!. \quad (24)$$

The function is therefore explicitly limited to $n \leq 10$. Fixing the first city removes rotational duplicates but does not otherwise alter the closed-tour optimization problem.

8 Optimization and Learning Procedure

The source is not a gradient-based learning system. Its “learning” behavior is entirely population based and derivative free. The adaptive quantities are mutation scales rather than gradients or neural-network weights.

8.1 Initialization and scale adaptation

The log-normal updates used in the mutation routines are consistent with the common design principle of self-adaptive ES step sizes: a positive scale is multiplied by a stochastic factor, which preserves positivity without requiring explicit projection onto the positive half-line. The one-step, n -step, and correlated branches differ in how many scale parameters are maintained and how those parameters are inherited.

8.2 Selection pressure and replacement

The default ES survivor configuration is offspring-only replacement. Because parents are discarded before survivor ranking, a parent that was excellent in the previous generation can disappear if its descendants do not preserve comparable quality. This is a standard consequence of (μ, λ) replacement and should be treated as an algorithmic characteristic rather than a bug.

The default TSP survivor configuration uses the same conceptual offspring-only scheme, but candidate selection is stochastic rather than a direct sort. That distinction affects how selection pressure is realized. In tournament mode the best of a sampled subset wins, whereas roulette-style selection biases probabilities continuously.

8.3 Objective thresholds

The ES stopping condition uses an absolute objective threshold. This makes sense for a benchmark function with a known target scale, but it is inherently objective-dependent. The TSP code uses fitness thresholds, so a condition such as $F \geq 6$ corresponds to a cost target of $C \leq 1/6$, not a route length of 6. The source also contains a six-city-specific cost threshold in a workflow wrapper; this is intentionally different from the generic fitness threshold mechanism and should not be conflated with it.

9 Implementation Details

9.1 Software dependencies

NumPy supplies arrays, random generators, vectorized distance calculations, numerical norms, and permutation operations. pandas is used mainly for tabular history export, and Matplotlib is used to generate convergence figures as part of the source’s artifact pipeline. JSON serialization provides machine-readable summaries. The source also contains imports for `argparse`, `csv`, and `os`, but these are not central to the executed workflow shown in the file.

SciPy and PyTorch are not required by the optimization kernels. They are probed opportunistically only for runtime metadata. This is a useful separation because it keeps the core computational path lightweight while allowing richer environment reporting when those libraries are present.

9.2 Copy semantics and mutable state

The ES code takes special care with NumPy arrays in strategy-parameter dictionaries. `_copy_parameter` makes explicit copies of arrays instead of returning references to parent state. This avoids aliasing when a child later mutates its strategy parameters. Such copying is especially important in a population implementation where arrays are mutable and are reused across generations.

9.3 Serialization

ES summaries exclude history arrays from the primary JSON summary and place them in CSV format when available. TSP results use a dedicated route CSV plus a JSON summary and optional history CSV. The result collector then builds a manifest identifying artifact type, relative path, size, and creation status before generating a ZIP archive.

This artifact separation is sensible for reproducibility: scalar metadata remains easy to parse, longer per-iteration histories are tabular, and binary or visualization files can be packaged separately. The current report does not reproduce any of those artifacts because the reporting requirement is code understanding rather than experiment output presentation.

9.4 Colab portability

The `is_colab` helper tests whether `google.colab` is importable. The final export stage triggers the Colab download API only when the notebook is actually running in Colab. Outside Colab it still produces the ZIP locally. This makes the script less dependent on a single execution environment.

10 Execution Workflow

The bottom portion of the source acts like a notebook driver. It sets `BASE_DIR`, `OUTPUT_ROOT`, and a seed, constructs the six-city validation distance matrix, runs the four bounded workflows, and then calls `run_workflows` followed by `collect_and_zip_results`.

Inside `run_workflows`, the program creates the output directory and seeds the global random sources. It records runtime metadata, stores the workflow inventory, and calls `make_validation_results`. It then writes the preserved original-scale configurations to JSON without executing those configurations by default.

The validation driver is layered. It first exercises all three ES mutation families, then executes the two ES workflow wrappers, then performs a small exact TSP consistency check, then executes the two TSP workflow wrappers. Finally, it attempts a forensic comparison against source-archive TSP output files if they happen to be present under the base directory. Missing

source archives are handled explicitly as a non-validation condition rather than treated as an error that invalidates the rest of the notebook.

This architecture is useful for a reconstruction task because the code does more than port the optimizer. It also creates checkpoints that test whether the reconstructed operators behave coherently and whether saved artifacts are internally consistent.

11 Design Decisions and Rationale

Several design choices are clear from the source even where their original motivation is not written as prose.

First, the source preserves multiple alternative operators rather than simplifying to one canonical algorithm. That is consistent with the reconstruction objective: the Python file acts as a configurable representation of the MATLAB workflows rather than a minimal implementation of a single algorithm.

Second, the default validation settings are deliberately smaller than the original-scale configurations. This suggests an intent to verify the computational path without imposing the cost of very large searches. The original-scale settings remain explicit and inspectable, which preserves provenance without making them the default execution burden.

Third, the code favors NumPy array operations for vector arithmetic and distance construction. In `euclidean_distance_matrix`, broadcasting computes all pairwise coordinate differences in a compact vectorized form before reducing across the coordinate dimension. This is computationally convenient for moderate instance sizes, although the resulting matrix occupies $O(n^2)$ memory.

Fourth, the source uses explicit validation functions instead of relying solely on successful execution. Route validity, finite values, dimension consistency, and exact small-instance optimization are all checked in dedicated routines. That is a strong engineering practice for a reconstruction whose main scientific risk is silent deviation from the original behavior.

The correlated ES branch requires a more cautious interpretation. The presence of an angle matrix and angle updates indicates an intention to retain correlated-mutation state, but the actual covariance passed to the random sampler is diagonal. Therefore the implemented behavior is narrower than a full correlated Gaussian ES. The code comment itself recognizes this and states that the exact behavior is being preserved.

12 Computational Considerations

12.1 Evolution Strategy cost

Let d be the ES dimension, μ the parent population size, and λ the number of children. One generation evaluates λ children after crossover and mutation, so the dominant objective-evaluation count is $O(\lambda)$ times the cost of the objective. For the Ackley function, a straightforward vector cost is $O(d)$ per candidate, giving approximately

$$T_{\text{Ackley,gen}} = O(\lambda d) \quad (25)$$

for objective evaluation alone. Mutation in the correlated branch also creates a $d \times d$ Gaussian perturbation matrix and therefore has higher per-child storage and sampling overhead than a scalar or diagonal-scale representation.

The ES configuration uses $\lambda = 7\mu$, since `nc_multiplier` is seven. This is a relatively offspring-heavy generation model: for every parent population of size μ , seven times as many children are generated before replacement. That increases evaluation cost per generation but can provide more candidates for selection.

12.2 TSP cost

The TSP cost function is $O(n)$ per route once the distance matrix is available. For a population of size N_p , evaluating the population is $O(N_p n)$. The distance matrix itself requires $O(n^2)$ storage and $O(n^2)$ work to construct from coordinates.

Permutation crossover and the mutation operators are generally $O(n)$ for one pair or one route, although the PMX and cycle implementations contain repeated position searches that can add constant-factor overhead and can approach $O(n^2)$ behavior in the worst case for naive search through parent arrays. The exact effective cost depends on the route length and branch taken.

The exact brute-force verifier is factorial:

$$T_{\text{exact}}(n) = O((n-1)!n), \quad (26)$$

when each permutation is evaluated with an $O(n)$ route-cost calculation. The explicit $n \leq 10$ limit is therefore a necessary safeguard for the intended validation role.

12.3 Memory behavior

The main persistent TSP memory object is the $n \times n$ distance matrix. Histories can also become significant if thousands of iterations are stored with full routes. In the source, each TSP history entry stores the best route for the iteration, giving $O(Tn)$ history storage across T iterations. ES history stores objective values and best vectors, leading to $O(Td)$ storage for best-variable history.

Because the default validation modes are bounded, this history cost is manageable. The original-scale configuration of 200,000 ES iterations would make history retention materially more expensive, which is one reason the source's option to disable history recording is technically relevant.

13 Reproducibility and Reliability

The source contains several positive reproducibility mechanisms. Explicit seeds are passed into each run, runtime versions are serialized, original-scale configurations are preserved, and validation results are written to machine-readable files. The repeated-run wrappers use deterministic seed offsets, making the set of runs reconstructible from a base seed and a count.

The architecture nevertheless has limits. Exact NumPy random-stream reproducibility can depend on the generator and software implementation, although the optimization kernels consistently use `numpy.random.default_rng`. Hardware acceleration does not directly affect these kernels because no GPU-based computation is used. Runtime metadata reduces ambiguity but does not pin package versions or provide a complete dependency lockfile.

The strongest reliability feature is exact validation on a tiny TSP instance. It does not establish global optimality for the evolutionary solver in general; instead, it provides a direct reference point for an instance small enough to enumerate exhaustively. That is the appropriate role for this type of validator.

A second reliability feature is the forensic comparison with saved route outputs when the original project files are available. The code recomputes costs using its Euclidean-distance convention and compares them with the stored values. The submitted file alone cannot complete this check because the referenced archive files are not included as hidden inputs. When they are absent, the source records a “NOT VALIDATED” state.

14 Limitations and Technical Considerations

14.1 Syntactic execution defect in the submitted file

The most immediate issue is purely syntactic. The source contains several top-level string literals before `from _future_ import annotations`. In Python, future imports must appear at the beginning of the module, after the module docstring and before ordinary executable statements. Because the additional strings are ordinary expressions rather than comments or the module docstring, the submitted file raises a `SyntaxError` before any workflow can run.

This defect is independent of the optimization algorithms. The body of the file can still be analyzed and, as an engineering matter, executed after relocating or removing the future import. The report treats the source as submitted and does not silently claim that it is directly runnable.

14.2 Correlation state without covariance rotation

As noted above, the correlated mutation branch maintains α but draws noise from $\text{diag}(\sigma)$. A full correlated ES would normally derive a non-diagonal covariance or rotation from the angle parameters. The current implementation therefore has a mismatch between the stored strategy state and the distribution actually sampled.

From a reconstruction perspective, this may reflect fidelity to the source behavior. From an algorithmic perspective, however, the operational mutation remains diagonal. A future extension that implements true correlated Gaussian mutation should change the covariance construction and validate that change independently.

14.3 Terminology encoded as identifiers

Several configuration identifiers contain spelling variants such as `tournament`, `roletwheel`, `trancation`, and `landa_va_mua`. The program handles them consistently internally, so they

are functional identifiers rather than runtime errors. Nevertheless, they reduce readability and interoperability with external documentation. A future cleanup could introduce standardized names while retaining aliases for backward compatibility.

14.4 Distance convention and TSPLIB semantics

The parser records `EDGE_WEIGHT_TYPE` but the active coordinate-based distance routine always computes raw Euclidean distances. The code therefore does not implement the full range of TSPLIB edge-weight conventions merely because it parses the field. This distinction matters if the parser is later applied to instances whose official distance semantics are not raw Euclidean coordinates.

14.5 Fitness reciprocity

Using $1/C$ is simple and monotone, but it can compress or amplify selection differences depending on the cost scale. It also makes threshold parameters less intuitive because the numerical scale is inverted. A direct minimization interface could be clearer in a more general solver, although the current reciprocal mapping is valid for positive costs.

14.6 No complete dependency environment specification

The metadata records versions at runtime, but the source does not provide a lockfile, package constraints, or an environment manifest. Reproducibility is therefore stronger at the algorithmic and artifact levels than at the exact environment level.

15 Overall Technical Assessment

The implementation is technically coherent as a research-oriented reconstruction harness. Its strongest feature is the separation between algorithmic primitives and validation/export infrastructure. The configuration is rich enough to represent multiple evolutionary strategies without duplicating the core search loops, and the validation checks target failure modes relevant to code translation: malformed routes, inconsistent dimensions, non-finite values, and disagreement with exact small-instance optima.

The ES side demonstrates several useful concepts: self-adaptive mutation scales, distinct one-step and n -step representations, crossover of both variables and strategy parameters, survivor replacement, bound handling, threshold termination, and repeated-run summaries. The TSP side adds the requirement for representation-aware genetic operators, since ordinary arithmetic recombination would not preserve permutation validity.

The main weaknesses are implementation-level rather than conceptual. The submitted file contains the future-import placement defect, the correlated ES branch does not use its angle matrix in the covariance, and the general TSP parser accepts an edge-weight type without using it to choose the distance convention. These points define the boundary between the intended design and the exact behavior of the submitted code.

For an academic software portfolio, the project demonstrates several useful competencies:

translating algorithms across languages, separating configuration from execution, recording reproducibility metadata, validating against exact reference solutions on small instances, and managing research artifacts. These strengths are best presented together with the limitations rather than as evidence of a more general optimizer than the implementation actually provides.

16 Conclusion

The submitted Python program reconstructs four evolutionary-optimization workflows across two problem classes: continuous benchmark optimization with Evolution Strategies and permutation optimization for the Traveling Salesman Problem. It is organized as a compact, reasonably modular research script. Configurations are represented as data classes, optimization kernels are separated from validation and serialization, and workflow wrappers preserve both bounded validation settings and the larger original configurations.

Mathematically, the ES component maintains a solution vector and mutation strategy parameters, applies stochastic recombination and log-normal scale adaptation, draws Gaussian perturbations, enforces box constraints, and selects survivors through either parent-plus-child or child-only replacement. The TSP component represents tours as permutations, evaluates closed-tour length through a distance matrix, converts cost to reciprocal fitness, uses permutation-preserving crossover and local mutations, and performs stochastic selection and survivor construction.

The reconstruction goes beyond a minimal algorithm translation by embedding reproducibility and forensic validation. Exact enumeration on a small TSP instance provides a useful ground truth, route structure is checked explicitly, and output artifacts include runtime metadata and manifests. At the same time, the submission should be interpreted precisely: the file has a Python syntax defect in the placement of the future import, the correlated ES state does not produce non-diagonal mutation covariance, and the TSPLIB edge-weight metadata does not control the Euclidean distance calculation.

Taken as a whole, the project is a credible example of research-oriented scientific programming centered on algorithm reconstruction, validation, and reproducible execution. Its value lies less in proposing a novel optimizer than in showing the ability to inspect an existing computational workflow, preserve its important algorithmic choices, make the reconstruction configurable and testable, and document where it is faithful, approximate, or limited.